Habit Shaper - Technical Solution

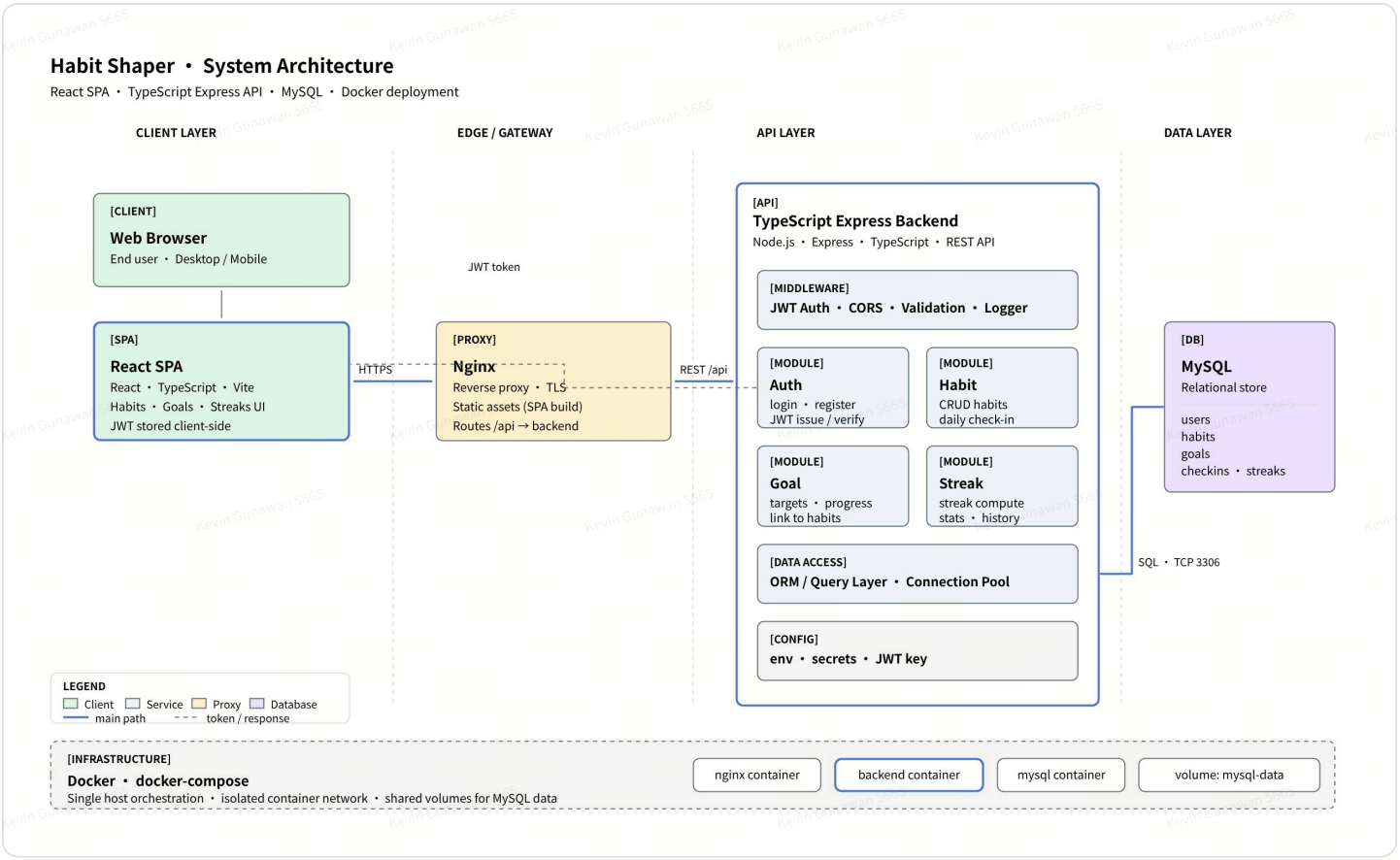
Overview

Habit Shaper is a lightweight, web-based application that helps users build positive habits and break negative ones through daily tracking and streak management. This document outlines the complete technical solution covering system architecture, database design, API contracts, frontend design, deployment strategy, and testing approach.

> ⭐ **Key Technical Decisions:**
>
> - **Frontend:** React (TypeScript) with modern hooks and context
> - **Backend:** Node.js + Express (TypeScript) REST API
> - **Database:** MySQL 8.x with relational schema
> - **Deployment:** Docker + docker-compose for single-host orchestration
> - **Auth:** JWT-based stateless authentication

System Architecture

The system follows a **three-tier architecture** with clear separation of concerns between presentation, business logic, and data layers, all orchestrated via Docker containers.



Component Breakdown

Component	Technology	Responsibility
Web Client	React 18 + TypeScript + Vite	SPA handling UI rendering, form inputs, state management, and API communication
Reverse Proxy	Nginx	TLS termination, static asset serving (SPA build), routing /api requests to backend
API Server	Node.js + Express + TypeScript	Business logic, authentication, input validation, habit/goal/streak management
Database	MySQL 8.x	Persistent relational storage for users, habits, goals, logs, and streaks
Infrastructure	Docker + docker-compose	Container orchestration, isolated network, volume persistence for MySQL data

---

Database Design

Entity Relationship Diagram

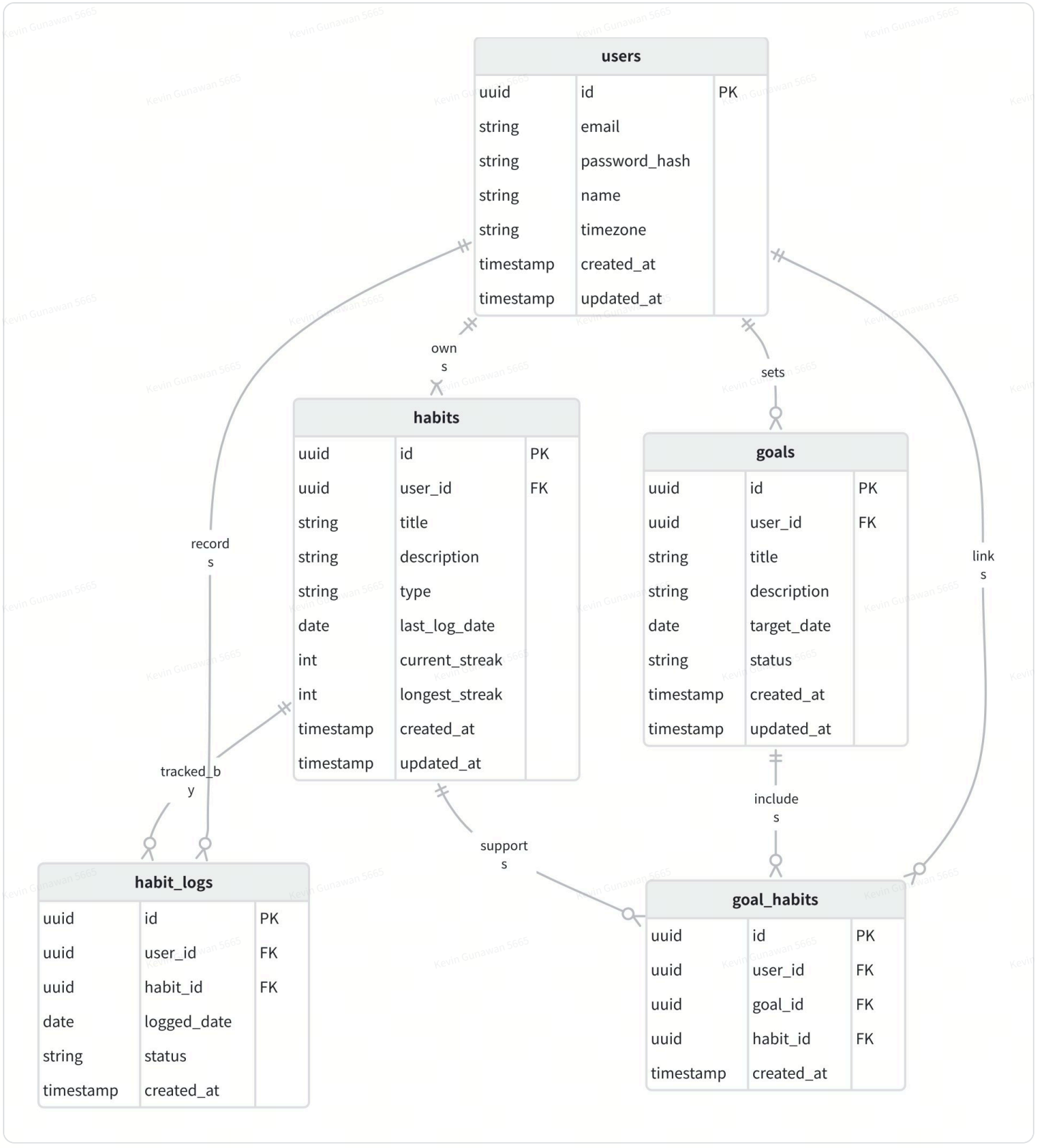


Table Definitions

`users`

Column	Type	Nullable	Constraints	Description
id	BIGINT	NO	PK, AUTO_INCREMENT	Unique user identifier

email	VARCHAR(255)	NO	UNIQUE INDEX	User email for authentication
password_hash	VARCHAR(255)	NO	-	bcrypt hashed password (12 rounds)
name	VARCHAR(100)	NO	-	Display name
timezone	VARCHAR(50)	NO	DEFAULT "UTC"	IANA home timezone (e.g., Asia/Jakarta). Used to determine "today" for streak calculations.
created_at	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	Account creation time
updated_at	TIMESTAMP	YES	ON UPDATE CURRENT_TIMESTAMP	Last modification time

`habits`

Column	Type	Nullable	Constraints	Description
id	BIGINT	NO	PK, AUTO_INCREMENT	Unique habit identifier
user_id	BIGINT	NO	FK → users.id, INDEX	Owner of the habit
title	VARCHAR(200)	NO	-	Habit name
description	TEXT	YES	-	Optional detailed description
type	ENUM("BUILDING","BREAKING")	NO	-	Whether building a new habit or breaking a bad one
last_log_date	DATE	YES	-	Most recent date user interacted with this habit (in user home TZ). BUILDING: last completion date. BREAKING: last relapse date.

current_streak	INT	NO	DEFAULT 0	Running streak count, updated on each mark action
longest_streak	INT	NO	DEFAULT 0	All-time best streak, updated when current_streak exceeds it
created_at	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	Creation time
updated_at	TIMESTAMP	NO	ON UPDATE CURRENT_TIMESTAMP	Last modification time

`habit_logs`

Column	Type	Nullable	Constraints	Description
id	BIGINT	NO	PK, AUTO_INCREMENT	Unique log entry identifier
user_id	BIGINT	NO	FK → users.id, INDEX	Owner of the log entry
habit_id	BIGINT	NO	FK → habits.id, INDEX	Associated habit
logged_date	DATE	NO	UNIQUE(habit_id, logged_date)	Date of the action in user’ s home timezone (one per day)
status	ENUM("COMPLETED","RELAPSED")	NO	-	COMPLETED = building done, RELAPSED = breaking failed. MISSED/CLEAN derived at read time.
created_at	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	UTC timestamp of when log was created

`goals`

Column	Type	Nullable	Constraints	Description
id	BIGINT	NO	PK, AUTO_INCREMENT	Unique goal identifier
user_id	BIGINT	NO	FK → users.id, INDEX	Owner of the goal

title	VARCHAR(200)	NO	-	Goal name
description	TEXT	YES	-	Optional detailed description
target_date	DATE	YES	-	Target completion date
status	ENUM('ACTIVE','COMPLETED','ABANDONED','PARTIALLY COMPLETED')	NO	DEFAULT 'ACTIVE'	Current status of the goal
created_at	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	Creation time
updated_at	TIMESTAMP	NO	ON UPDATE CURRENT_TIMESTAMP	Last modification time

`goal_habits` (Junction Table)

Column	Type	Nullable	Constraints	Description
id	BIGINT	NO	PK, AUTO_INCREMENT	Unique record identifier
user_id	BIGINT	NO	FK → users.id, INDEX	Owner of the goal-habit link
goal_id	BIGINT	NO	FK → goals.id, UNIQUE(goal_id, habit_id)	Associated goal
habit_id	BIGINT	NO	FK → habits.id	Associated habit
created_at	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	Link creation time

Key Indexes

- `idx_habits_user_id` on `habits(user_id)` — quick lookup of all habits for a user
- `idx_habit_logs_streak` on `habit_logs(habit_id, logged_date DESC, status)` — fast range query for weekly dashboard calendar view
- `idx_habit_logs_user_id` on `habit_logs(user_id)` — quick lookup of all logs for a user

- `idx_goals_user_id` on `goals(user_id)` — quick lookup of all goals for a user
- `idx_goal_habits_goal_id` on `goal_habits(goal_id)` — join performance
- `idx_goal_habits_user_id` on `goal_habits(user_id)` — quick lookup of all goal-habit links for a user

---

API Design

The backend exposes a RESTful API with JSON request/response bodies. All endpoints except auth routes require a valid JWT in the `Authorization: Bearer <token>` header.

Authentication Endpoints

Method	Endpoint	Request Body	Response
POST	`/api/auth/register`	`{ email, password, name }`	`{ user, token }` — 201 Created
POST	`/api/auth/login`	`{ email, password }`	`{ user, token }` — 200 OK
GET	`/api/auth/me`	-	`{ user }` — 200 OK (requires auth)

Habit Endpoints

Method	Endpoint	Request Body	Response
GET	`/api/habits`	-	`{ habits[] }` — list user's habits with streaks
POST	`/api/habits`	`{ title, description?, type }`	`{ habit }` — 201 Created
GET	`/api/habits/:id`	-	`{ habit, logs[] }` — habit detail with log history
PUT	`/api/habits/:id`	`{ title?, description?, type? }`	`{ habit }` — 200 Updated
DELETE	`/api/habits/:id`	-	204 No Content
POST	`/api/habits/:id/mark`	`{ date? }` (defaults to today)	`{ log, streak }` — mark completion/clean day

Goal Endpoints

Method	Endpoint	Request Body	Response
GET	`/api/goals`	-	`{ goals[] }` — list user's goals
POST	`/api/goals`	`{ title, description?, target_date?, habit_ids[] }`	`{ goal }` — 201 Created
GET	`/api/goals/:id`	-	`{ goal, habits[] }` — with linked habits
PUT	`/api/goals/:id`	`{ title?, description?, target_date?, status?, habit_ids[]? }`	`{ goal }` — 200 Updated
DELETE	`/api/goals/:id`	-	204 No Content

Streak/Tracking Endpoint

Method	Endpoint	Query Params	Response
GET	`/api/habits/:id/stats`	`from`, `to` (date range)	`{ current_streak, longest_streak, total_logs, calendar[] }`

Error Response Format

All errors follow a consistent structure:

Code block

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human-readable description",
    "details": [{ "field": "email", "message": "must be a valid email" }]
  }
}

```

Frontend Design

Technology Stack

- **React 18** with functional components and hooks
- **TypeScript** for type safety
- **Vite** as build tool (fast HMR, optimized production builds)
- **React Router v6** for client-side routing
- **Axios** for HTTP requests with interceptors
- **React Context + useReducer** for global auth state
- **CSS Modules** or **Tailwind CSS** for styling

Page Structure

Route	Component	Description
`/login`	`LoginPage`	Email/password login form
`/register`	`RegisterPage`	New account registration
`/dashboard`	`DashboardPage`	Overview of today's habits, active streaks, and goals progress
`/habits`	`HabitsListPage`	All habits with streak indicators and mark buttons
`/habits/:id`	`HabitDetailPage`	Habit history calendar, streak stats, edit form
`/goals`	`GoalsListPage`	All goals with linked habits and progress
`/goals/:id`	`GoalDetailPage`	Goal details, linked habits, status management

Key Components

- `AuthProvider` — Context provider managing JWT token, user state, login/logout
- `ProtectedRoute` — HOC that redirects unauthenticated users to `/login`
- `HabitCard` — Reusable card showing habit name, type badge, streak count, and mark action

- `StreakCalendar` — Visual calendar grid showing completion/clean days
- `GoalProgress` — Displays goal with linked habit streak summaries

State Management Strategy

Global State (Context)

- Auth token and user profile
- Loading/error states for auth

Local State (Component)

- Form inputs and validation
- Habit list, goal list (fetched per page)
- Modal/dialog open state

Authentication and Security

Authentication Flow

1. User submits email + password to `/api/auth/login`
2. Server validates credentials against bcrypt hash
3. Server generates a JWT (HS256, 24h expiry) containing `{ userId, email }`
4. Client stores token in `localStorage` and attaches it to all subsequent requests via Axios interceptor
5. Backend middleware extracts and verifies JWT on every protected route
6. On token expiry, client receives 401 and redirects to login

Security Measures

> 💭 **Security Checklist to be considered:**
>
> - Passwords hashed with **bcrypt** (cost factor 12)
> - JWT secret stored in environment variable, never committed to source
> - Input validation via **express-validator** or **zod** on all endpoints
> - Rate limiting on auth endpoints (e.g., 5 attempts per minute per IP)
> - CORS configured to allow only the frontend origin
> - SQL injection prevention via parameterized queries (ORM)
> - XSS prevention via React's default escaping + Content-Security-Policy headers

---

Streak Logic — last_log + Stored Counters

Design Approach: last_log + Stored Counters

Streaks are maintained using three columns stored directly on the `habits` table: `last_log_date`, `current_streak`, and `longest_streak`. There is **no cron job** and **no backward log scanning**. Streak counters are updated on each write (mark/relapse) and read directly on each fetch with a staleness check.

> ⭐ **Core Insight:** Store streak state (`last_log_date`, `current_streak`, `longest_streak`) directly on the `habits` row. On **write**: compare `last_log_date` with today/yesterday to decide increment vs. reset. On **read**: apply a staleness check — for building habits, if `last_log_date < yesterday`, display streak = 0. For breaking habits, clean streak = `daysBetween(last_log_date, today)`. `habit_logs` serves purely as an **audit trail** for the weekly dashboard.

Code block

```
// On Read — get displayed streak for any habit
function getDisplayedStreak(habit: Habit, user: User): number {
  const today = getCurrentDate(user.timezone);
  const yesterday = subtractDays(today, 1);

  if (habit.type === "BUILDING") {
    // Staleness check: if last completion was before yesterday, streak has lapsed
    if (!habit.last_log_date || habit.last_log_date < yesterday) return 0;
    return habit.current_streak;
  } else {
    // BREAKING: clean streak = days since last relapse
    if (!habit.last_log_date) return daysBetween(habit.created_at, today);
    return daysBetween(habit.last_log_date, today);
  }
}

```

Breaking Habits — last_log_date Approach

For **breaking habits**, `last_log_date` stores the date of the last relapse. The clean streak is calculated passively:

- `last_log_date` = date of the last relapse (or NULL if never relapsed)
- Displayed streak = `daysBetween(last_log_date, today)` in user's timezone

- When user relapses: set `last_log_date = today`, `current_streak = 0`
- Absence of relapse IS the clean day — streak grows passively every day without any background job

Code block

```
// Breaking habit: mark relapse
function markBreakingRelapse(user: User, habit: Habit) {
  const today = getCurrentDate(user.timezone);

  // Calculate current clean streak before resetting
  const currentCleanStreak = habit.last_log_date
    ? daysBetween(habit.last_log_date, today)
    : daysBetween(habit.created_at, today);

  // Save longest streak if current exceeds it
  const newLongest = Math.max(habit.longest_streak, currentCleanStreak);

  // Record relapse event (audit trail for weekly dashboard)
  upsertLog(habit.id, user.id, today, "RELAPSED");

  // Reset: last_log_date = today, current_streak = 0
  updateHabit(habit.id, {
    last_log_date: today,
    current_streak: 0,
    longest_streak: newLongest
  });

  return { currentStreak: 0, longestStreak: newLongest };
}

```

Building Habits — last_log_date Comparison

For **building habits**, the streak counter is updated by comparing `last_log_date` against today and yesterday:

- User marks `COMPLETED` during the day via the API
- If `last_log_date == yesterday` → streak continues: `current_streak += 1`
- If `last_log_date < yesterday` or NULL → streak broken: `current_streak = 1`
- If `last_log_date == today` → already marked, no-op (idempotent)

Code block

```
// Building habit: mark as completed for today
```

```
function markBuildingComplete(user: User, habit: Habit) {
  const today = getCurrentDate(user.timezone);
  const yesterday = subtractDays(today, 1);

  // Already marked today — idempotent
  if (habit.last_log_date === today) {
    return { currentStreak: habit.current_streak, longestStreak:
habit.longest_streak };
  }

  // Calculate new streak
  let newStreak: number;
  if (habit.last_log_date === yesterday) {
    newStreak = habit.current_streak + 1; // consecutive day
  } else {
    newStreak = 1; // missed ≥1 day or first ever mark
  }

  const newLongest = Math.max(habit.longest_streak, newStreak);

  // Record completion event (audit trail for weekly dashboard)
  upsertLog(habit.id, user.id, today, "COMPLETED");

  // Update habit counters
  updateHabit(habit.id, {
    last_log_date: today,
    current_streak: newStreak,
    longest_streak: newLongest
  });

  return { currentStreak: newStreak, longestStreak: newLongest };
}

```

Timezone Handling

💡 **How it works:** Each user stores their IANA timezone (e.g., `Asia/Jakarta`) in the `users` table. When the user accesses the app, `getCurrentDate(user.timezone)` computes "today" and "yesterday" relative to the user's local time using `Intl.DateTimeFormat`. This means a user in Jakarta (UTC+7) and a user in New York (UTC-4) each see correct streak values — timezone is resolved at request time.

```
function getCurrentDate(timezone: string): string {
  return new Intl.DateTimeFormat("en-CA", {
    timeZone: timezone,
    year: "numeric", month: "2-digit", day: "2-digit"
  }).format(new Date()); // Returns "YYYY-MM-DD"
}

```

Weekly Streak Dashboard

The weekly dashboard provides a 7-day grid view of all user habits. It requires only **2 database queries** (all habits + all logs for the week) and in-memory mapping:

Code block

```
function getWeeklyDashboard(user: User, weekStart: string) {
  const today = getCurrentDate(user.timezone);
  const weekEnd = addDays(weekStart, 6);

  // Query 1: all user habits
  const habits = query("SELECT * FROM habits WHERE user_id = ?", [user.id]);

  // Query 2: all logs for the week (sparse — audit trail only)
  const logs = query(`
    SELECT habit_id, logged_date, status FROM habit_logs
    WHERE user_id = ? AND logged_date BETWEEN ? AND ?
  `, [user.id, weekStart, weekEnd]);

  // Build grid: for each habit, derive status per day
  return habits.map(habit => ({
    habit,
    currentStreak: getDisplayedStreak(habit, user),
    days: buildWeekDays(habit, logs, weekStart, today)
  }));
}

function getDayStatus(habit, log, date, today): string {
  if (date > today) return "FUTURE";
  if (date === today) {
    if (habit.type === "BUILDING")
      return log?.status === "COMPLETED" ? "COMPLETED" : "PENDING";
    else
      return log?.status === "RELAPSED" ? "RELAPSED" : "CLEAN_SO_FAR";
  }
  // Past days
  if (habit.type === "BUILDING")
```

```
      return log?.status === "COMPLETED" ? "COMPLETED" : "MISSED";
    else
      return log?.status === "RELAPSED" ? "RELAPSED" : "CLEAN";
  }

```

API Endpoint: `GET /api/dashboard/weekly?week_start=2026-06-02`

Goal Management

Business Rules

1. Each goal **must be linked to at least 1 habit** at creation time
2. A goal can be linked to multiple habits (many-to-many via `goal_habits`)
3. A habit can belong to multiple goals
4. Goal progress is derived from the combined streak performance of its linked habits

Goal Progress Calculation

Code block

```
interface GoalProgress {
  goal: Goal;
  linkedHabits: Array<{
    habit: Habit;
    currentStreak: number;
    isOnTrack: boolean; // streak > 0 today
  }>;
  overallScore: number; // percentage of linked habits on track
}

```

Deployment Architecture

Docker Compose Configuration

The application is deployed as a multi-container Docker setup:

Code block

```yaml
# docker-compose.yml
version: '3.8'
services:
  nginx:
    image: nginx:alpine
    ports: ["80:80", "443:443"]
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
      - ./frontend/dist:/usr/share/nginx/html
    depends_on: [backend]

  backend:
    build: ./backend
    environment:
      - DB_HOST=mysql
      - DB_PORT=3306
      - DB_NAME=habit_shaper
      - JWT_SECRET=${JWT_SECRET}
    depends_on:
      mysql:
        condition: service_healthy

  mysql:
    image: mysql:8.0
    environment:
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
      - MYSQL_DATABASE=habit_shaper
    volumes:
      - mysql_data:/var/lib/mysql
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  mysql_data:

```

Container Responsibilities

Nginx Container

- Serves React SPA static files

Backend Container

- Node.js + Express app

MySQL Container

- Persistent data via named volume

- Proxies `/api/*` to backend
- Handles TLS termination
- Runs database migrations on startup
- Exposes port 3000 internally
- Health check ensures readiness
- Auto-initializes schema on first run

Testing Strategy

Testing Pyramid

Level	Tool	Coverage Target	Examples
Unit Tests	Jest + ts-jest	Business logic, streak calculation, validators	Streak calculator, input validation, JWT utilities
Integration Tests	Supertest + Jest	API endpoint behavior with real DB	Auth flow, CRUD operations, streak updates after marking
Frontend Tests	React Testing Library	Component rendering, user interactions	Login form submission, habit card mark action, route guards
E2E Tests	Cypress / Playwright	Critical user journeys end-to-end	Register → Create habit → Mark → Verify streak → Create goal

---

Project Structure

Code block

```
habit-shaper/
├── docker-compose.yml
├── nginx.conf
├── frontend/
│   ├── package.json
│   ├── tsconfig.json
│   ├── vite.config.ts
│   ├── src/
│   │   ├── main.tsx
│   │   ├── App.tsx
│   │   ├── api/              # Axios instance + API calls
│   │   ├── components/       # Shared UI components
│   │   ├── contexts/         # AuthContext
│   │   ├── pages/            # Route-level page components
```

```
│   │   ├── hooks/          # Custom hooks (useAuth, useHabits)
│   │   └── types/          # TypeScript interfaces
│   └── Dockerfile
├── backend/
│   ├── package.json
│   ├── tsconfig.json
│   ├── src/
│   │   ├── index.ts        # Express app entry
│   │   ├── config/         # Environment config
│   │   ├── middleware/     # Auth, validation, error handler
│   │   ├── routes/         # Route definitions
│   │   ├── controllers/    # Request handlers
│   │   ├── services/       # Business logic (streak calc)
│   │   ├── models/         # ORM models / DB queries
│   │   └── utils/          # JWT helpers, date utils
│   └── Dockerfile
└── db/
    ├── init.sql            # Initial schema DDL
    └── migrations/         # Incremental migrations

```

Non-Functional Requirements

Performance

- API response time < 200ms (p95)
- Frontend initial load < 3s on 3G
- Database queries optimized with proper indexing
- Connection pooling for MySQL (pool size: 10)

Reliability

- MySQL data persisted via Docker volumes
- Graceful shutdown handling in Node.js
- Health check endpoints for monitoring
- Structured JSON logging for debugging

Risks and Mitigations

Risk	Impact	Mitigation
Streak calculation becomes expensive at scale	Slow page loads for active users	Streaks read from `habits` row (O(1)). No log scanning. `last_log_date` +

		`current_streak` + `longest_streak` stored directly on the habit.
JWT token theft via XSS	Account compromise	CSP headers, HttpOnly cookies (future), short token TTL
MySQL single point of failure	Data loss on container crash	Named volumes with host mount, regular backup cron job
Timezone discrepancies in streak tracking	Incorrect streak counts	Home timezone stored on users table; `logged_date` stored as plain DATE computed in user's home timezone at action time via `Intl.DateTimeFormat`